

Analyzing Earthquakes from a Simulated Dynamic Fault Model

Date: April 13, 2026

First Name: Dana

Last Name: Adhis

Student ID: 1007097801

Introduction

What is an Earthquake?

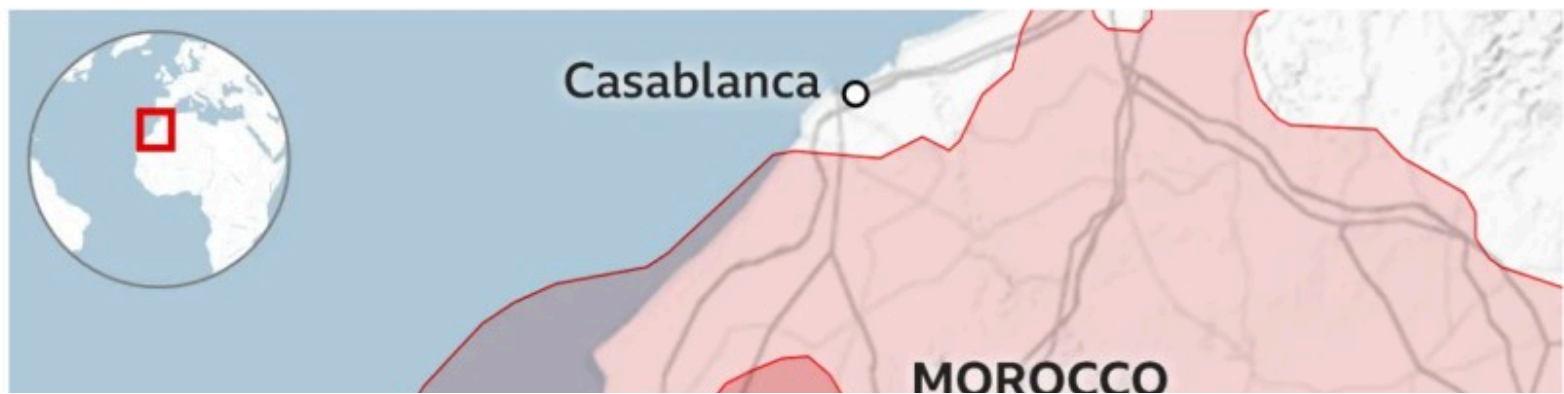
Earthquakes are among nature's most fatal forces, causing substantial harm to citizens, infrastructure, and economies. They occur when Earth's tectonic plates shift, separate, or collide, releasing tons of energy in the process.

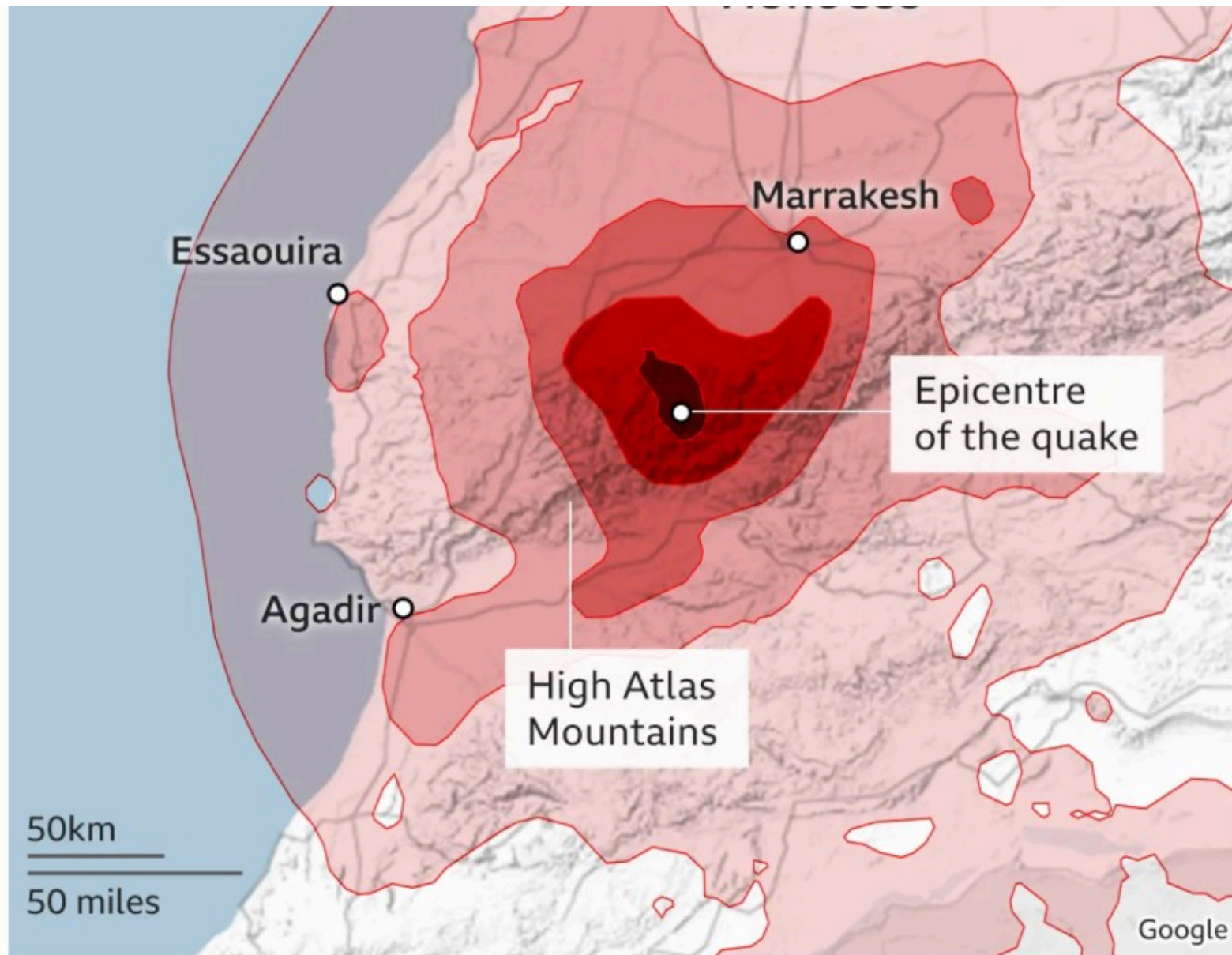
The 2023 earthquake in Morocco, my home country, is a notable example of the destruction earthquakes can cause and illustrates the importance of studying earthquakes. It was triggered by the collision between the African and Eurasian plates, where the African plate continuously pushes northward, building up stress along the Atlas Mountains. Witnessing the devastation this caused made the importance of understanding earthquakes and protecting vulnerable communities from them.

Figure 1 shows the aftermath of the 2023 Morocco earthquake, while **Figure 2** illustrates the intensity of shaking across the affected regions relative to the epicenter.



Figure 1: Aftermath of the 2023 Earthquake event in Morocco (Morocco Earthquake, 2023).





Intensity of the quake

■ Severe ■ Very strong ■ Strong ■ Moderate ■ Light

Source: USGS

BBC

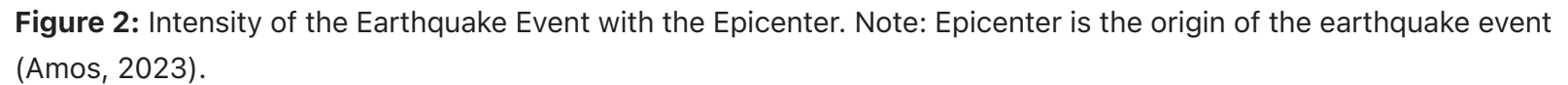


Figure 2: Intensity of the Earthquake Event with the Epicenter. Note: Epicenter is the origin of the earthquake event (Amos, 2023).

Methods to Analyze Earthquakes

Analyzing earthquakes through numerical models can answer many field-based questions that seismologists may have such as which physical parameters induce certain conditions, what drives particular stress states, and, in general, how to test hypotheses that can then be compared against observational or experimental work. This is imperative given the limitations of observational studies, which are caused by limited instrument resolution, insufficient geodetic coverage, and a lack of data capture in certain regions. Therefore this makes numerical modeling an essential complement to field-based analysis.

Important Definitions

- The **moment magnitude** is a scale used to measure the size of the earthquake; it is a logarithmic scale (demonstrated by Figure 3).
- The **seismic moment** is how much energy is released by an earthquake.
- The **frequency-magnitude distribution (FMD)** of earthquakes represents the relationship between the number of earthquakes and their magnitude (size). This is following the Gutenberg-Richter (G-R) law: $\log N = a - bM$ where N is the number of earthquakes, a is a constant representing the frequency of earthquakes --> a high 'a' indicates many seismic events are occurring, and b is a constant, but it represents a ratio of small to large earthquake events. It is essentially the slope of the frequency of the earthquakes and their magnitude.
- The **magnitude completeness (Mc)** is the smallest magnitude event that a seismic station can detect precisely.

Earthquake frequency and destructive power

The left side of the chart shows the magnitude of the earthquake and the right side represents the amount of high explosive required to produce the energy released by the earthquake. The middle of the chart shows the relative frequencies.

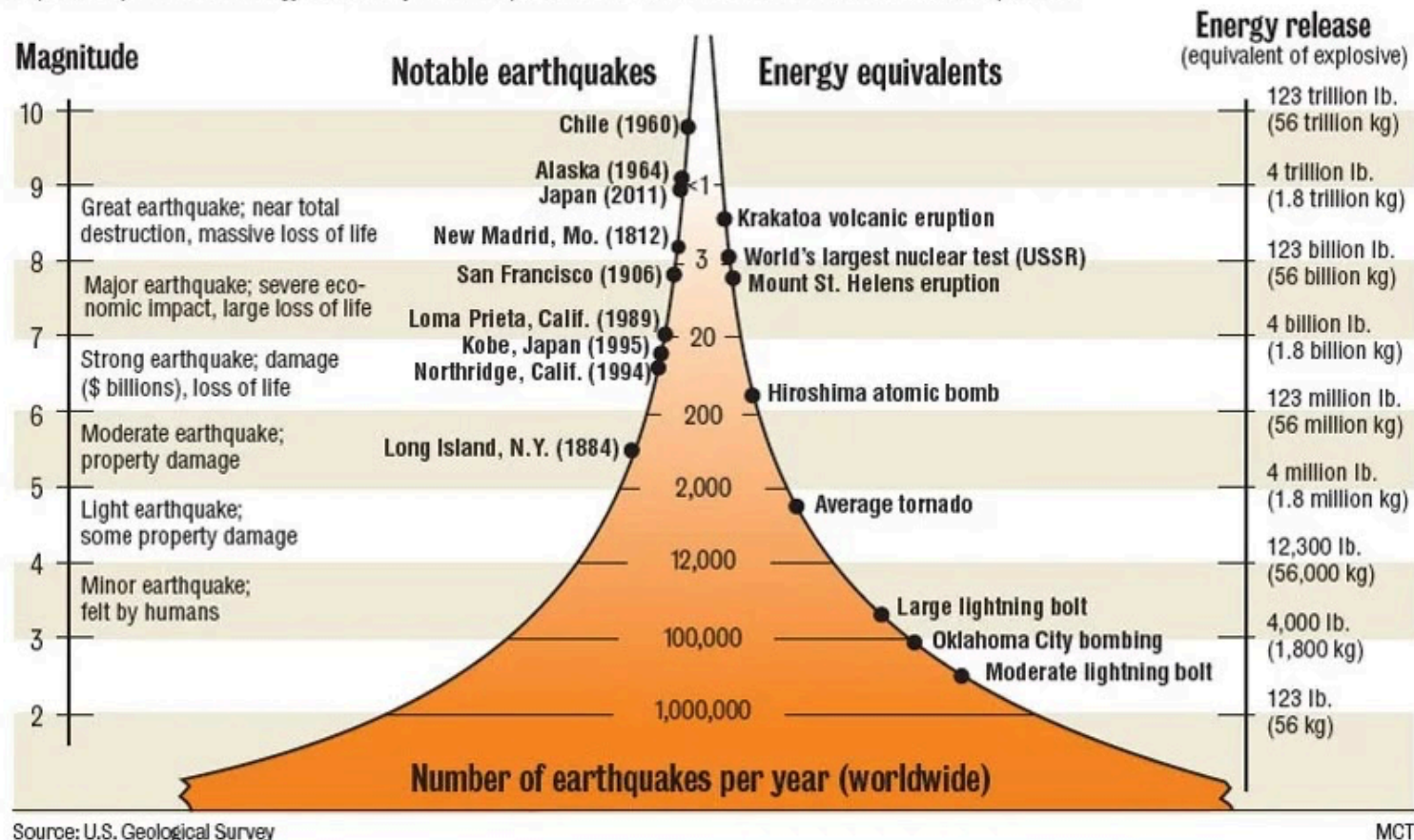


Figure 3: Richter scale displaying information regarding the moment magnitude and the approximate energy release (GeologyIn, 2015).

Numerical Simulated Seismic Catalogues

Analyzing a simulated earthquake catalog is similar to analyzing real seismic data. In this Jupyter Notebook, we will

explore and visualize data from a simulated catalog called the **Ecatalogue**. We can examine where earthquakes occur along the fault, their size (moment magnitude), the energy they release (seismic moment), and how earthquake sizes are distributed across the catalog (FMD).

The data come from a numerical model called the Boundary Integral Cycle of Earthquakes (BICycle). Think of it as a virtual earthquake machine; it simulates how earthquakes naturally develop and recur over long periods along a fault (Lapusta et al., 2000; Ito & Kaneko, 2023). The model covers a 1,200-meter stretch of the fault in the seismogenic zone, the region where earthquakes occur.

To understand how the fault is simulated, imagine the fault divided into several small grid patches, called rupture cells. Each cell along the fault has specific physical properties such as strength, stress, and slip speed (depending on the model). Once an earthquake initiates, the first cell fails, and the rupture cascades into neighboring cells. However, the cascade does not continue if the stress is insufficient to sustain the propagation (The domino effect will not continue if there is not enough stress present to impact the next cells). **Figure 4** displays an example of a fault that is cut into a grid, each square represents a cell, and its color indicates how much the cell slipped (Li et al., 2021).

The fault in the simulation that we will explore has a total length of 1,200 meters divided into 16,384 cells, giving each cell a size of approximately 0.073 meters. A mainshock is defined here as an event where the majority of the fault has ruptured. Specifically, we define a mainshock as any event in which at least 80% of the fault ruptures, corresponding to approximately 13,107 cells failing in a single event.

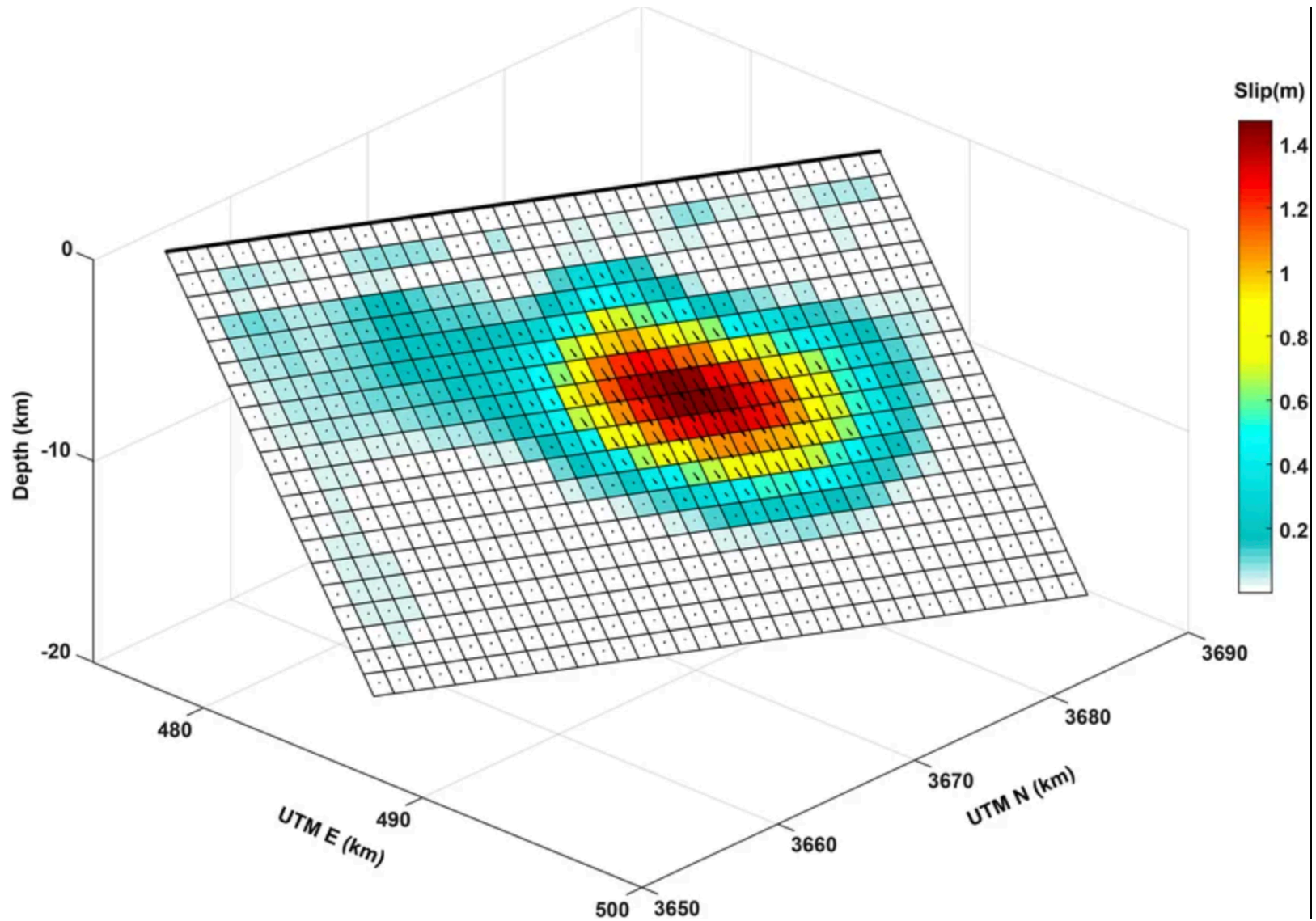


Figure 4: Three-dimensional finite-fault slip distribution with each square on the grid is 1 x 1 km (Li et al., 2021).

Objective of this Tutorial

The main objective of this tutorial is to provide a brief hands-on overview of the data that can be extracted and analysed using only the Ecatalogue. Through a series of plots, we can confirm validations and better understand the seismic patterns of a simulated fault.

1. Filter the data, only including the events from a specific time window.
2. Compute the magnitude of completeness (M_c).
3. Compute the b-value using the Maximum Likelihood Method and Linear Regression.
4. Plot a figure of the fault position against time, highlighting mainshock events.
5. Plot moment magnitude against time.
6. Plot cumulative seismic moment against time.
7. Plot the frequency-magnitude distribution using the Maximum Likelihood Method and Linear Regression.
8. Plot the relationship of the seismic moment and rupture cells.
9. Plot the relationship of the distribution of ruptured events and the number of earthquake events

1. Data Filtering - Ecatalogue at a Glance

1.1 Importing Libraries

We need to import the Python libraries for data cleaning, loading, calculation as well as visualisation.

```
In [1]: # We import pandas for data loading and cleaning,  
# numpy for calculations, matplotlib for plotting,  
# and scipy for statistical analysis  
# such as linear regression.  
  
# We also use estimate_mc_maxc function  
# from the seismostats library to analyze the catalog and  
# estimate the magnitude of completeness ( $M_c$ ).  
  
import pandas as pd  
import numpy as np  
import matplotlib
```

```
import matplotlib.pyplot as plt
from scipy.stats import linregress
from seismostats.analysis import estimate_mc_maxc
from seismostats.analysis import ClassicBValueEstimator
from seismostats import Catalog
```

1.2 Loading and Filtering Data

Let us take a look at the structure of the Ecatalogue. The first cell loads the data and the second cell is the table down below which displays each column and what it represents.

```
In [2]: # Load the catalog
A = pd.read_csv('Ecatalogue', sep='\s+', header=None)

# Extract columns
eventID      = A[0].values
xevstartcell = A[1].values
rupcellno    = A[2].values
tsevstart    = A[3].values
tsevend      = A[4].values
tevstart     = A[5].values
tevend       = A[6].values
Mom          = A[7].values
Mw           = A[8].values

# Conversion constants
yrinsec = 31557600 # seconds in a year (365.25 days)
# size of each rupture cell in metres (~0.073 m)
hcell   = 1200 / 16384
# convert event start time from seconds to years
years   = tevstart / yrinsec
```

```
In [3]: df = {
# Data column names
    'Data_IDs': ['eventID', 'xevstartcell', 'rupcellno', 'tsevstart', 'tsevend',
                 'tevstart', 'tevend', 'Mom', 'Mw'],
# Description of the data columns
```

```

    'Description': [
        'Sequential event identifier',
        'Location of the event in the grid',
        'Number of cells ruptured',
        'Timestep at which the event begins',
        'Timestep at which the event ends',
        'Start time of the event in seconds',
        'End time of the event in seconds',
        'Seismic moment (N·m)',
        'Moment magnitude'
    ]
}

# Create a DataFrame from the dictionary
df_cols = pd.DataFrame(df)
display(df_cols)

```

	Data_IDs	Description
0	eventID	Sequential event identifier
1	xevstartcell	Location of the event in the grid
2	rupcellno	Number of cells ruptured
3	tsevstart	Timestep at which the event begins
4	tsevend	Timestep at which the event ends
5	tevstart	Start time of the event in seconds
6	tevend	End time of the event in seconds
7	Mom	Seismic moment (N·m)
8	Mw	Moment magnitude

Before proceeding with the analysis, we need to first clean the data. The first mainshock in the catalog is removed from the analysis as it is an artefact of the loading process used to initialise the fault. To do this, we identify the year in which the first mainshock occurs and remove all data up to and including that point.

```

In [4]: # Find the first mainshock and remove all data up to and including it
first_mainshock      = np.argmax(rupcellno > 13107)
first_mainshock_time = years[first_mainshock]
print(f"First mainshock in year: {first_mainshock_time:.7f}")

# Apply filter
filter_data = years > first_mainshock_time

# Extract filtered columns
new_years      = years[filter_data]
new_xevstartcell = xevstartcell[filter_data]
new_rupcellno   = rupcellno[filter_data]
new_tevstart    = tevstart[filter_data]
new_tevend      = tevend[filter_data]
new_Mom         = Mom[filter_data]
new_Mw          = Mw[filter_data]

# Save filtered data as a new DataFrame
A_filtered = pd.DataFrame({
    'years':      new_years,
    'xevstartcell': new_xevstartcell,
    'rupcellno':   new_rupcellno,
    'tevstart':    new_tevstart,
    'tevend':      new_tevend,
    'Mom':         new_Mom,
    'Mw':          new_Mw
})

# Print and display results
print(f"Total events: {len(new_years)}")
A_filtered.to_csv('Ecatalogue_filtered.csv', index=False)

display(A_filtered.head())

```

First mainshock in year: 1.9929882

Total events: 127

	years	xevstartcell	rupcellno	tevstart	tevend	Mom	Mw
0	1.993142	9900	494	6.289879e+07	6.289879e+07	6.827060e+10	1.156156
1	1.993921	23304	502	6.292337e+07	6.292337e+07	9.159615e+10	1.241251
2	2.058206	9961	500	6.495205e+07	6.495205e+07	1.102357e+11	1.294881
3	2.058222	10485	683	6.495256e+07	6.495256e+07	1.955357e+11	1.460817
4	4.430088	23432	488	1.398029e+08	1.398029e+08	1.215881e+11	1.323261

2. Minor Calculations

2.1 Calculating the Magnitude Completeness (Mc) using the Maximum Curvature (MAXC) using SeismoStats

Let us start off with some minor calculations to look into the catalog, we can determine the magnitude completeness using the method known as the Maximum Curvature. The Maximum Curvature method functions by finding the magnitude bin with the highest number of earthquakes in the histogram. This peak marks the point at which the catalog (and even our simulated catalog) begins to miss smaller events; thus, it is only complete to a certain level (Rehman et al., 2016).

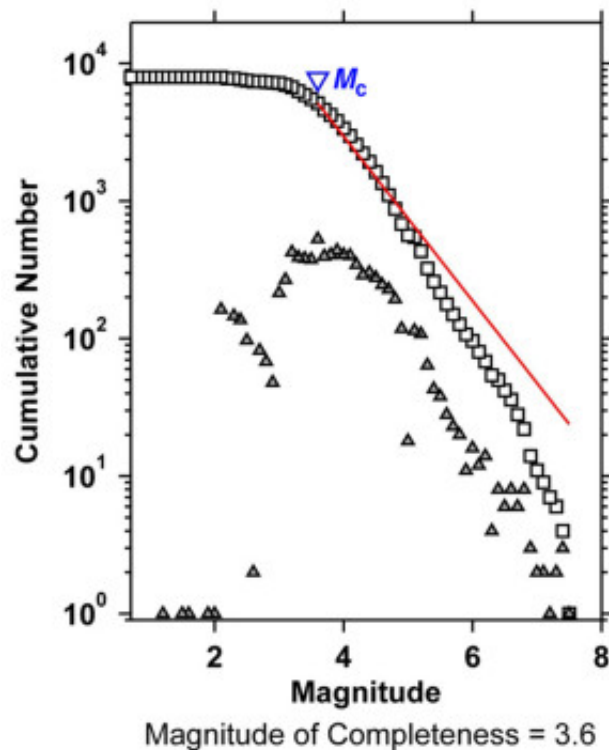


Figure 5: The MAXC technique being used to determine the magnitude completeness (Rehman et al., 2016).

We utilise SeismoStats, an open-source library, to compute the magnitude of completeness. The method was tested both with and without the +0.2 correction factor to determine the effect on the resulting M_c estimate. The correction factor exists because the Maximum Curvature method tends to underestimate M_c . However, since this is a simulated dataset, both cases are tested to determine which provides a more appropriate estimate. It is important to compute these estimates (with and without correction), as the resulting plots in the following section will allow us to assess which method provides a more suitable fit through visual inspection.

```
In [5]: # Using SeismoStats to estimate the magnitude of completeness (Mc)
# using the maximum curvature method
cat = Catalog({'magnitude': A_filtered['Mw'].values})
cat.estimate_mc_maxc(fmd_bin=0.1, correction_factor=False)
#I removed the correction factor
print(f"Mc = {cat.mc}")
```

```
Mc = 1.4
```

```
In [6]: cat.estimate_mc_maxc(fmd_bin=0.1)
print(f"Mc = {cat.mc}")
```

```
Mc = 1.5999999999999999
```

2.2 Calculating the b-value using the Maximum Likelihood Method (MLM) using SeismoStats

The Maximum Likelihood Method (MLM) is widely used in seismicity analysis and is the accepted standard so far for estimating the b-value. Here we again utilise SeismoStats to compute the b-value from the catalog.

```
In [7]: # We estimate the b-value using the ClassicBValueEstimator
# from SeismoStats,
# ClassicBValueEstimator implements the maximum likelihood
# estimation (MLE) method to calculate the b-value
# of a seismic catalog.
# Round magnitudes to the nearest 0.1
# to match the binning used for Mc.

magnitudes = np.round(A_filtered['Mw'].values / 0.1) * 0.1
estimator = ClassicBValueEstimator()
estimator.calculate(magnitudes=magnitudes, mc=cat.mc, delta_m=0.1)
print(f"b-value = {estimator.b_value:.3f} ± {estimator.std:.3f}")
```

```
b-value = 1.131 ± 0.170
```

2.3 Calculating the b-value using Linear Regression

Linear regression is also used as an alternative method for estimating the b-value. Although it is less robust than the Maximum Likelihood Method, it is applied here for comparison purposes as it provides a useful check on the consistency of the results. This is particularly valuable for simulated datasets, where using multiple methods provides greater clarity and a more comprehensive understanding of the data.

```
In [8]: # Compute cumulative frequency:
```

```

# Sort magnitudes in ascending order
sorted_Mw = np.sort(magnitudes)
cumulative_frequency = np.arange(len(sorted_Mw), 0, -1)
log_cumulative_frequency = np.log10(cumulative_frequency)

# Mask to filter magnitudes greater than or equal to Mc
mc_filter = sorted_Mw >= cat.mc
# The Mc was already calculated above
# using the estimate_mc_maxc function,
# so we can directly use cat.mc here.

# Filtered magnitudes for linear regression
mc_filtered_Mw = sorted_Mw[mc_filter]
mc_filtered_log_cf = log_cumulative_frequency[mc_filter]

slope, intercept = linregress(mc_filtered_Mw, mc_filtered_log_cf)[:2]
b_value = -slope

print(f"b-value = {b_value:.4f}")

```

b-value = 0.7895

3. Data Visualization

3.1 Plot the Fault Position against Time, Highlighting the Mainshock Events.

This plot shows the time and location along the fault where a mainshock occurred and the events. Moreover, this plot is beneficial for future work, as it allows a deeper analysis of stress plots to understand how the fault behaves before a mainshock is triggered and to see how seismicity moves along the fault.

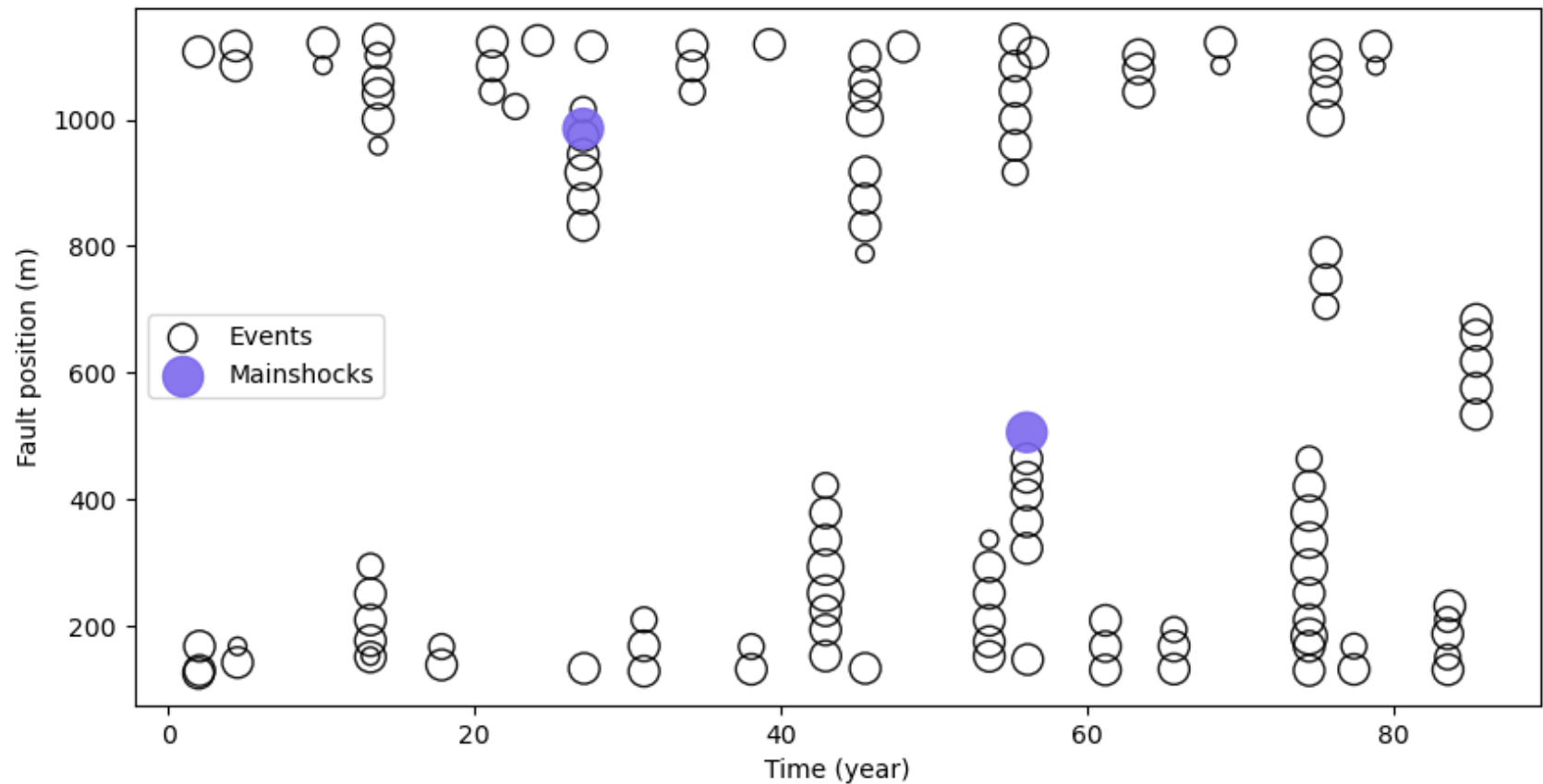
We can plot the fault position against time using the filtered dataset, highlighting mainshock events where they occur. The positions are plotted in the same manner in which the simulation describes the fault, using the physical cell positions along the fault plane. For simplification: fault_positions represents the actual positions along the fault, starting at -600 metres where the seismogenic zone begins and ending at 1800 metres, with each step equal to one cell width of approximately 0.073 metres.

In [9]: *# Plotting the fault space with the highlighted mainshocks.*

```
fault_positions = np.arange(-600, 1800 + hcell, hcell)
xevstartpos = fault_positions[new_xevstartcell.astype(int)]

# Seperating the seismic magnitudes into bins for
# plotting the different sizes of the data.
Mwtype = np.digitize(new_Mw, bins=[-np.inf, 0, 1, 2, 3, np.inf])
highlight_mask = new_rupcellno > 13107 #Mainshock identifiers

#Plotting
fig, ax = plt.subplots(figsize=(10, 5))
ax.scatter(new_tevstart[~highlight_mask]/yrinsec, xevstartpos[~highlight_mask],
           s=Mwtype[~highlight_mask] * 50, facecolor='none', edgecolors='k',
           alpha=0.9, label='Events')
ax.scatter(new_tevstart[highlight_mask]/yrinsec, xevstartpos[highlight_mask],
           s=Mwtype[highlight_mask] * 50, facecolor='mediumslateblue', alpha=0.9,
           edgecolors='mediumslateblue', label='Mainshocks')
plt.legend()
ax.set_ylabel('Fault position (m)')
ax.set_xlabel('Time (year)');
```



```
In [10]: # Finding the time of the mainshocks in years
mainshock_years = new_tevstart[highlight_mask]/yrinsec
for yr in mainshock_years:
    print(f"Mainshock at year: {yr:.4f}")
    recurrence = mainshock_years[1]-mainshock_years[0]
print(f"Recurrence interval between mainshocks: {recurrence:.4f} years")
```

Mainshock at year: 27.0884

Mainshock at year: 56.0265

Recurrence interval between mainshocks: 28.9382 years

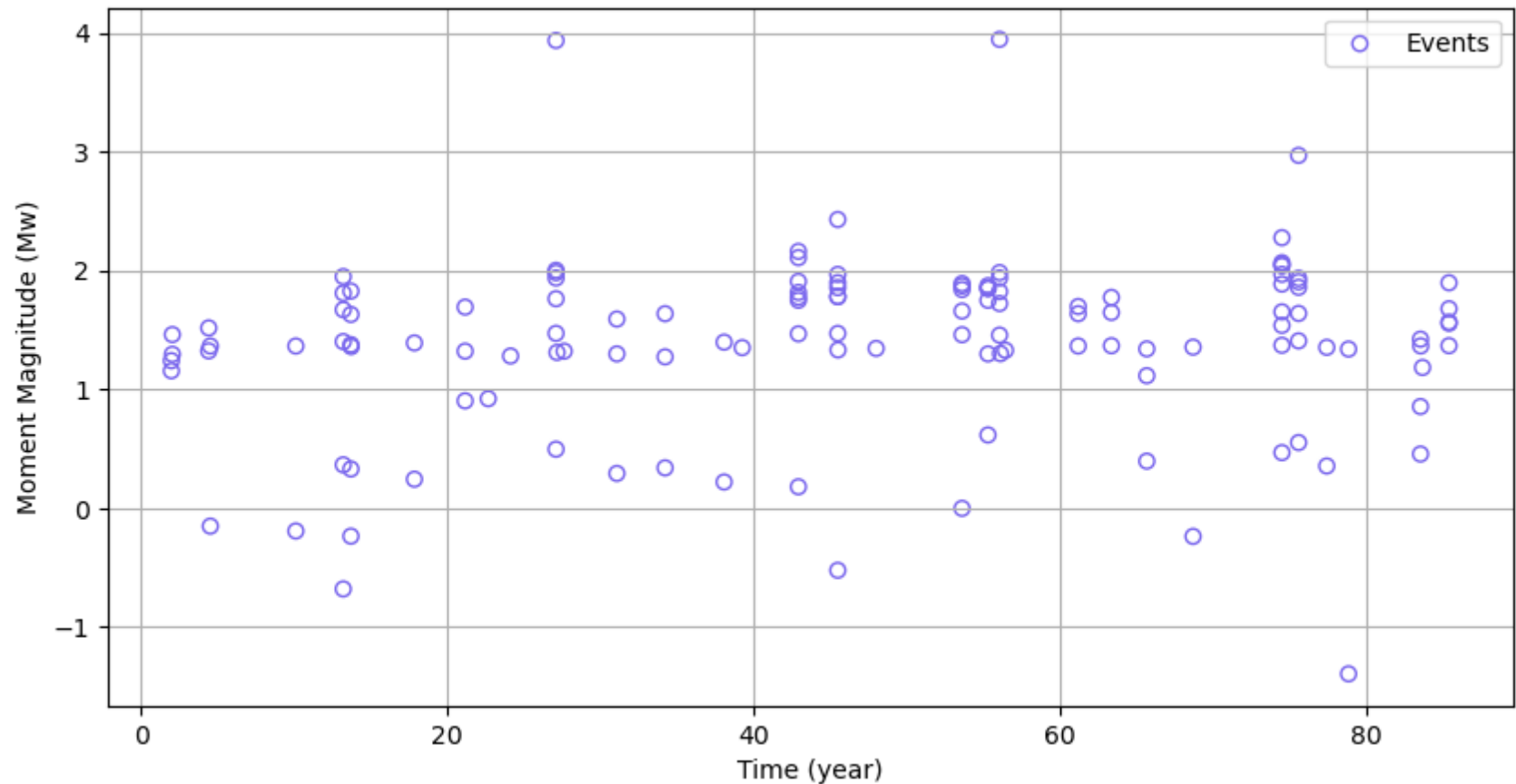
From the figure above, it is clear that there are two main shock events that appear to recur approximately every 29 years. We are also able to calculate the timing.

3.2 Plot the Moment Magnitude against Time.

Plotting the moment magnitude against time shows the distribution of earthquake sizes that have occurred on the fault over the duration of the simulation.

```
In [11]: #Plotting
fig, ax = plt.subplots(figsize=(10, 5))
# Scatter plot of all events showing the
# distribution of magnitudes over time.
scatter = ax.scatter(new_tevstart/yrinsec, new_Mw, edgecolor='mediumslateblue', facecolors='none',
ax.set_ylabel('Moment Magnitude (Mw)')
ax.set_xlabel('Time (year)')
# Added grid to see the distribution a bit more clearly.
plt.grid(True)
plt.legend()

plt.show()
```

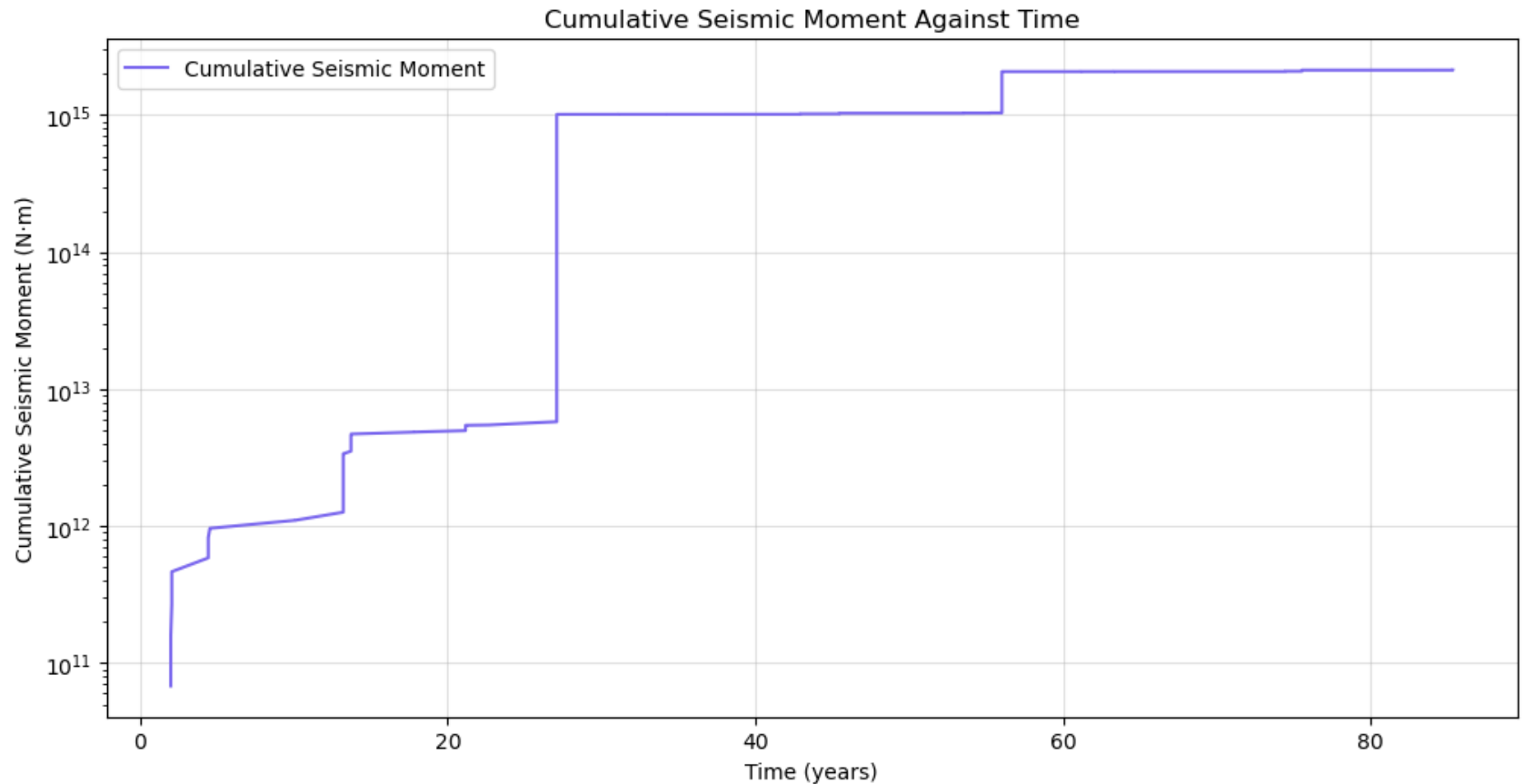


3.3 Plot the Cumulative Seismic Moment against Time.

Plotting the cumulative seismic moment is useful as each sudden jump indicates a large earthquake event, while each flat section represents a period of relative seismic stability.

```
In [12]: # Cumulative sum of the seismic moment to capture how it evolves over time.
cumulative_Mom = np.cumsum(new_Mom)
#Plotting
plt.figure(figsize=(10, 5))
plt.plot(new_years, cumulative_Mom, color='mediumslateblue', label='Cumulative Seismic Moment')
plt.yscale('log')
plt.legend()
```

```
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.xlabel('Time (years)')
plt.ylabel('Cumulative Seismic Moment (N·m)')
plt.title('Cumulative Seismic Moment Against Time')
plt.show()
```



3.4 Plot the Frequency-Magnitude Distribution using the Maximum Likelihood Method and Linear Regression.

Having computed the magnitude of completeness and b-value in the previous section, we can now plot the frequency-magnitude distribution to visualise the results.

```

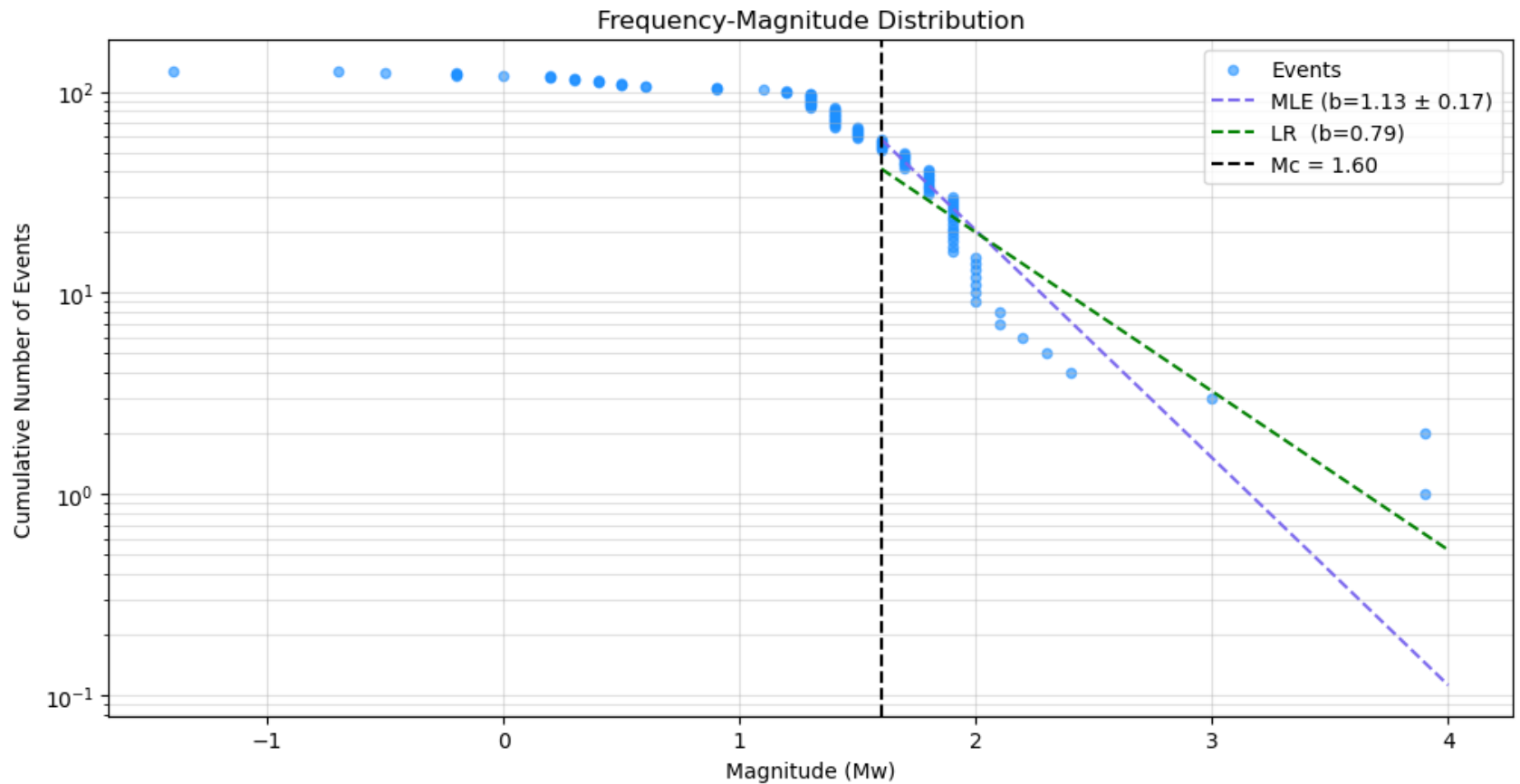
In [13]: # Linear Regression (LR) Method to estimate the b-value and plot the GR line.
a = intercept #recall the a constant in the GR law.
mag_range     = np.arange(cat.mc, sorted_Mw.max() + 0.1, 0.1) # Bins
fit_lr = 10 ** (a - b_value * mag_range) # GR Formula

# ----- #

# Maximum Likelihood Estimation (MLE) Method to
# estimate the b-value and plot the GR line.
# Calculate a-value for MLE
a_value_mle = np.log10(len(magnitudes[magnitudes >= cat.mc])) + estimator.b_value * cat.mc
fit_mle = 10 ** (a_value_mle - estimator.b_value * mag_range)

#Plotting
plt.figure(figsize=(10, 5))
plt.scatter(sorted_Mw, cumulative_frequency, s=20,
            color='dodgerblue', alpha=0.6, label='Events')
plt.plot(mag_range, fit_mle, color='mediumslateblue',
         linestyle='--',
         label=f'MLE (b={estimator.b_value:.2f} ± {estimator.std:.2f})')
plt.plot(mag_range, fit_lr, 'g--', label=f'LR (b={b_value:.2f})')
plt.axvline(cat.mc, color='k', linestyle='--', label=f'Mc = {cat.mc:.2f}')
plt.yscale('log')
plt.legend()
plt.grid(True, which='both', alpha=0.4)
plt.tight_layout()
plt.xlabel('Magnitude (Mw)')
plt.ylabel('Cumulative Number of Events')
plt.title('Frequency-Magnitude Distribution')
plt.show()

```



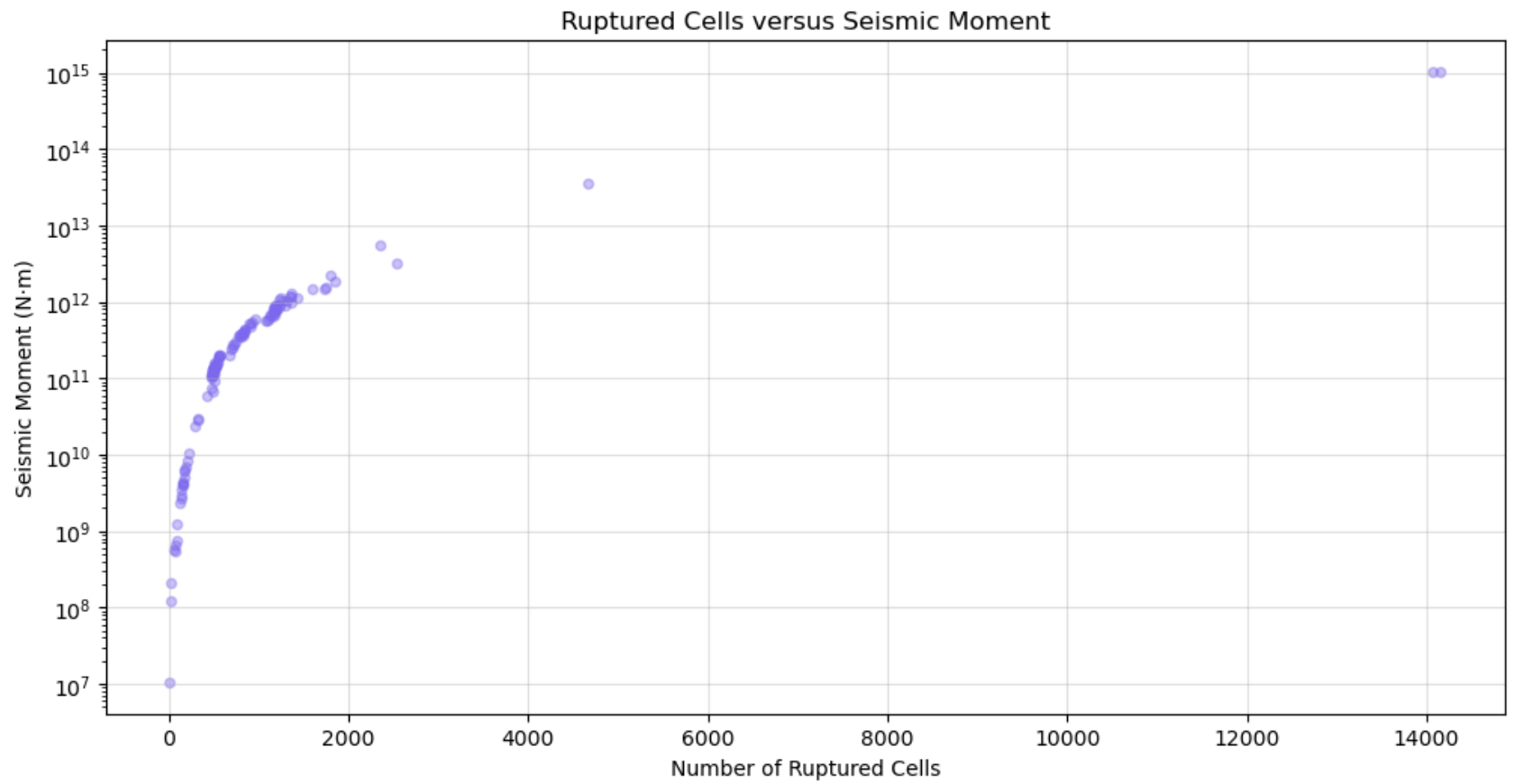
The frequency-magnitude distribution shows that the magnitude completeness is $M_c = 1.60$ (using a correction of 0.2). The overall data follows the expected Gutenberg-Richter trend. If we focus on the MLE method, the b-value is 1.13 ± 0.17 , which implies an abundance of relatively small earthquakes compared to large ones. However, the linear regression method gives a lower b-value of 0.79, which is not unusual; in some cases, using the linear regression model tends to underestimate when there are few large events, as in this case.

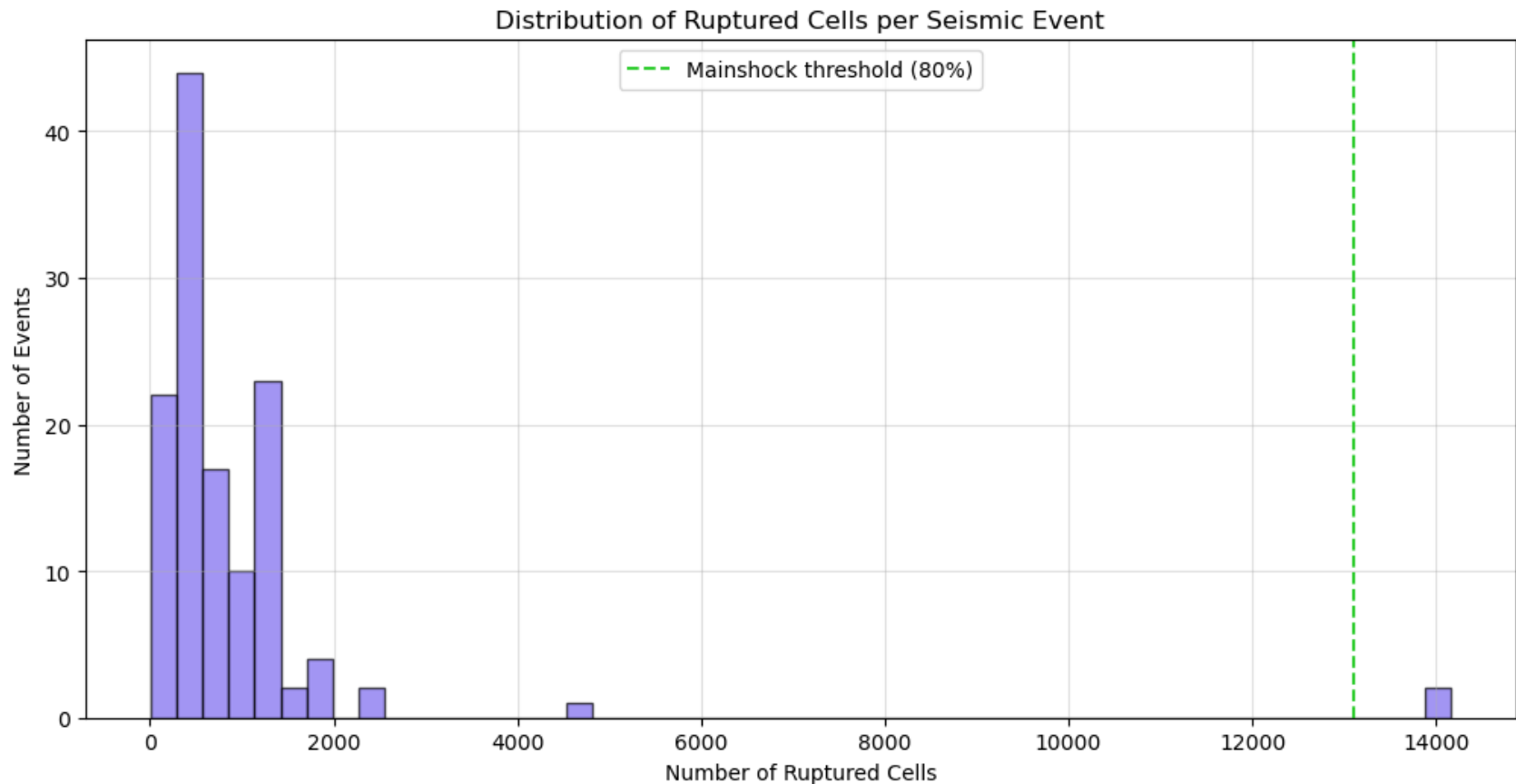
One thing to note is that the results display realistic seismic behavior, as shown across the plots. The data below M_c are irregular, as expected, given that below this threshold, the catalog is incomplete, and smaller events are not fully captured. Overall, both methods provide useful estimates of the b-value and contribute to a better understanding of the seismic behavior of the simulated fault.

3.4 Extra Figures and Analysis using the Rupture Cell Number

As a preliminary validation, we present additional plots that provide a broader overview of the catalog. The seismic moment against ruptured cells and the number of events against ruptured cells both serve to characterise the nature of the simulated catalog. For instance, in this dataset the relatively low number of mainshocks is reflected in the distribution of rupture cell counts, where large rupture events are infrequent compared to smaller ones.

```
In [14]: #Plotting the relationship between the number  
# of ruptured cells and the seismic moment.  
plt.figure(figsize=(10, 5))  
plt.scatter(new_rupcellno, new_Mom, s=20, alpha=0.4, color='mediumslateblue')  
plt.yscale('log')  
plt.grid(True, alpha=0.4)  
plt.tight_layout()  
plt.xlabel('Number of Ruptured Cells')  
plt.ylabel('Seismic Moment (N·m)')  
plt.title('Ruptured Cells versus Seismic Moment')  
plt.show()  
  
# ----- #  
  
#Plotting the relationship between the distribution  
# of ruptured cells and the number of events.  
plt.figure(figsize=(10, 5))  
plt.hist(new_rupcellno, bins=50, edgecolor='k', alpha=0.7,  
        color='mediumslateblue')  
plt.axvline(13107, color='limegreen', linestyle='--',  
           label='Mainshock threshold (80%)')  
plt.legend(loc = 'upper center')  
plt.grid(True, which='both', alpha=0.4)  
plt.tight_layout()  
plt.xlabel('Number of Ruptured Cells')  
plt.ylabel('Number of Events')  
plt.title('Distribution of Ruptured Cells per Seismic Event')  
plt.show()  
  
print(f'# of data >= 13107 ruptured cells: {np.count_nonzero(new_rupcellno >= 13107)}')
```





of data $\geq$ 13107 ruptured cells: 2

4. Conclusion

Analyzing the Ecatalogue provides insight, particularly when using SeismoStats, which considerably simplifies the statistical analysis. The figures in this Jupyter Notebook show that the dataset is dominated by small-magnitude earthquakes, with large events occurring infrequently (because the simulation ended before we saw the next mainshock). These large events appear to follow extended periods of seismic inactivity, with a recurrence interval of about 30 years before the next large earthquake (by speculation).

There remains considerable further analysis, however, outside the Ecatalogue file. Future work could involve

investigating stress data from the simulation to examine the mechanisms that trigger large events at specific times. It would also be valuable to examine maximum slip velocity (V_{max}) plots, which would provide insight into the timing and location of slip episodes along the fault. Overall, the Ecatalogue provides an excellent preliminary foundation for characterizing the seismicity of the simulated fault before proceeding to more detailed and critical analysis.

5. References

1. Amos, J. (2023, September 9). Morocco earthquake: Such magnitude unusual for country. Bbc.com; BBC News. <https://www.bbc.com/news/world-africa-66762984>
2. Bender, B. (1983). Maximum likelihood estimation of b values for magnitude grouped data. Bulletin of the Seismological Society of America, 73(3), 831–851. <https://doi.org/10.1785/bssa0730030831>
3. GeologyIn. (2015, January 30). Using the Richter Scale to Measure Earthquakes. Geology In. <https://www.geologyin.com/2015/01/using-richter-scale-to-measure.html>
4. Ito, R., & Kaneko, Y. (2023). Physical Mechanism for a Temporal Decrease of the Gutenberg-Richter b-Value Prior to a Large Earthquake. Journal of Geophysical Research Solid Earth, 128(12). <https://doi.org/10.1029/2023jb027413>
5. Lapusta, N., Rice, J. R., Yehuda Ben-Zion, & Zheng, G. (2000). Elastodynamic analysis for slow tectonic loading with spontaneous rupture episodes on faults with rate- and state-dependent friction. Journal of Geophysical Research, 105(B10), 23765–23789. <https://doi.org/10.1029/2000jb900250>
6. Li, K., Li, Y., Tapponnier, P., Xu, X., Li, D., & He, Z. (2021). Joint InSAR and Field Constraints on Faulting During the Mw 6.4, July 23, 2020, Nima/Rongma Earthquake in Central Tibet. Journal of Geophysical Research Solid Earth, 126(9). <https://doi.org/10.1029/2021jb022212>
7. Morocco earthquake: Stories of heartbreak emerge in Tamaloukte | IFRC. (2023, September 18). Ifrc.org. <https://www.ifrc.org/article/morocco-earthquake-stories-heartbreak-emerge-tamaloukte>
8. Rehman, F., El-Hady, S. M., Atef, A. H., & Harbi, H. M. (2016). Seismic hazard assessment of Western Coastal

Province of Saudi Arabia: deterministic approach. *Earthquake Science*, 29(5), 299–309.

<https://doi.org/10.1007/s11589-016-0164-1>

9. Wang, X., Li, J., Feng, A., & Sornette, D. (2025). Estimating Magnitude Completeness in Earthquake Catalogs: A Comparative Study of Catalog-Based Methods. *Journal of Geophysical Research: Solid Earth*, 130(9).

<https://doi.org/10.1029/2025jb031441>